

Efficient Real-Time Recognition Model of Plant Diseases for Low-Power Consumption Platform

Songyun Deng , Wanneng Wu , Kunlin Zou , Hai Qin , Lekai Cheng , and Qiaokang Liang , *Member, IEEE*

Abstract—Recognition and early warning of plant diseases is one of the keys to agricultural disaster prevention and mitigation. Deep learning-based image recognition methods give us a new idea for plant disease identification. Due to the harsh conditions in agricultural environment, recent research has focused on exploring ways to lightweight the recognition model for deployment on low-power devices. In this article, we propose an efficient and feature-guided real-time plant disease recognition model with a multiclassifier architecture, specifically designed for low-power devices. By comparing with other advanced methods, our model reaches the state of the art in the combined metrics of recognition accuracy, the number of parameters, and inference speed. First, we propose AMI-NanoNet based on Roofline theory to significantly reduce the number of parameters and computational complexity. This model can achieve 99.8343% accuracy on PlantVillage by using a feature-guided curriculum learning with stepwise training strategy. Moreover, we design another training strategy suitable for lightweight ensemble models. Based on this strategy, our model only needs to integrate the classifiers at the end of the network to achieve 99.8708% identification accuracy, and it hardly increases the number of operations and parameters of the network. Extensive evaluations on this dataset demonstrate the effectiveness of our ensemble learning method. Furthermore, we tested our proposed methods on another dataset from other domains to validate its applicability to different scenarios. Overall, our research provides a basis for rapid and intelligent identification of plant diseases.

Impact Statement—Real-time image recognition of plant diseases is a promising emerging technology. However, existing recognition methods are difficult to be used in real agricultural environment due to the lack of datasets, high model computation cost, and insufficient identification accuracy. This article innovatively constructs an efficient and lightweight network with a multiclassifier structure, along with a feature-guided training strategy to overcome these

challenges. By comparing other advanced methods, our model can achieve higher recognition accuracy with lower computational cost. Extensive evaluations have shown that this technique can be used for efficient real-time identification of plant diseases and demonstrates applicability to other datasets in different scenarios. This will help the AI-based plant disease identification technology to be more widely used in practical agricultural production.

Index Terms—AMI-NanoNet, ensemble learning, Jetson Nano, lightweight network, plant disease identification.

I. INTRODUCTION

WITH the development of human society, plant diseases are increasingly posing a serious threat to global food security and causing a massive reduction in food production [1], [2]. Therefore, rapid diagnosis of plant diseases can save planting costs and drastically reduce food losses. Plant leaves are generally used to visually distinguish plant categories, so most plant disease research focuses on plant leaf images [3], [4]. In the past, the image recognition of plant diseases mainly relied on manually summarizing features [6]. Nevertheless, these conventional techniques exhibit a significant recognition error rate in diverse shooting scenarios and heavily depend on human expertise. Along with the advance of artificial intelligence methods, machine learning methods such as deep learning have been applied to the identification of plant leaf diseases, and it has been proved that these methods have a higher identification accuracy and stronger robustness than traditional ways [4]. Therefore, plant disease identification methods based on deep learning not only showcase great significance but also a promising market perspective.

Although deep learning-based plant disease identification methods achieve promising performance in multiple plant disease datasets [4] and the data collection process can now be automated [5], in the field of botany, the standard way to identify a type of plant disease is still based on the chemical biology methods provided by the plant pathology laboratory [7], [8]. Even for some neural network models with good recognition effects, it is nonetheless difficult for researchers to accurately explain the specific reasons for the classification results obtained by these models [9], [10], [11], [12]. On the other hand, in the general agricultural or forestry environment, the identification of plant diseases often requires a considerable area of investigation. Besides, the image recognition of plant diseases is affected by conditions, such as illumination changes, background complexity, and hardware equipment used for identification [13], [14]. These conditions pose challenges to the recognition speed,

Manuscript received 5 May 2023; revised 20 July 2023 and 16 August 2023; accepted 20 August 2023. Date of publication 23 August 2023; date of current version 14 May 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62073129 and Grant U21A20490, in part by the Hunan Provincial Natural Science Foundation of China under Grant 2022JJ10020, in part by the Project of the National Key Research and Development Program of China under Grant 2021YFC1910402, and in part by the Scientific Research Project of Hunan Education Department of China under Grant 21B0330. This paper was recommended for publication by Associate Editor Guilherme Nelson DeSouza upon evaluation of the reviewers' comments. (Corresponding author: Qiaokang Liang.)

Songyun Deng, Kunlin Zou, Hai Qin, and Lekai Cheng are with the College of Electrical and Information Engineering, Hunan University, Changsha 410082, China (e-mail: songyun@hnu.edu.cn; kunlin@hnu.edu.cn; qinhai@hnu.edu.cn; lekai@hnu.edu.cn).

Wanneng Wu is with the School of Electrical and Information Engineering, Changsha University of Science and Technology, Changsha 410114, China (e-mail: wuwanneng@csust.edu.cn).

Qiaokang Liang is with the National Engineering Research Center for Robot Vision Perception and Control, College of Electrical and Information Engineering, Hunan University, Changsha 410082, China (e-mail: qiaokang@hnu.edu.cn).

Digital Object Identifier 10.1109/TAI.2023.3307662

accuracy, and application scope of the plant disease recognition model.

In order to improve the recognition effect of the model and deal with the lack of datasets, some recent works propose to use enhanced training datasets or modify the loss function to improve the applicability of the model to a specific dataset [9], [15], [16], [17], [18], [19], [20]. Hence, these methods are often only applicable to a specific dataset, and these complex models are often difficult to apply to real-time recognition scenarios. Besides, in terms of the lightweight of recognition networks, some recent studies have proposed a variety of new lightweight networks, which have few parameters and better recognition performance [11], [21], [22], [23], [24], [25], [26], [27], [28], [29]. Most of these models use attention mechanism, transfer learning, model fusion, etc., to improve the efficiency of network recognition while reducing the amount of computation. However, some works pointed out that for a specific device, the smaller the model calculation volume does not necessarily lead to the faster calculation speed [30], [31]. In addition, these models discriminate disease labels directly without considering the deep relationships among labels (such as differences in plant categories, growing environments, etc.). This mode is different from the experience of human recognition methods, which brings great obstacles to the evidence-based model discrimination. Moreover, the identification errors caused by this factor also hinder the practical use of deep learning-based plant disease identification models.

In this article, we propose a solution for low-computing hardware devices in view of the above research shortcomings. To summarize, the contributions of this research are as follows.

- 1) Based on the Roofline theory, we propose attention mechanism in inception (AMI) modules A and B by combining the feature extraction advantages of multiscale structures and attention mechanisms with the low computational advantages of inverted residual structures. By utilizing these two modules, we are able to construct the AMI-NanoNet with a low number of convolutional layers for low-power devices, such as Jetson Nano [32]. Compared with other lightweight networks, the proposed model can achieve higher accuracy in plant disease identification, while requiring a small number of parameters and offering fast inference speed.
- 2) By introducing the idea of curriculum learning, we improve the traditional way of training deep learning models. Our proposed method [feature-guided curriculum learning (FGCL)] enhances the robustness of training by increasing the difficulty of the network learning samples gradually to avoid premature overfitting. Compared with traditional methods, our method has higher stability and is 0.018% higher than the former in the average macro F1 score of cross-validation.
- 3) We propose a novel ensemble learning method based on plant categories to further improve the classification accuracy and reliability. This method passes the Kappa-Error diagram test and finally improves the accuracy of our model to 99.8708%. Further comparison with the state-of-the-art plant disease classification models indicates

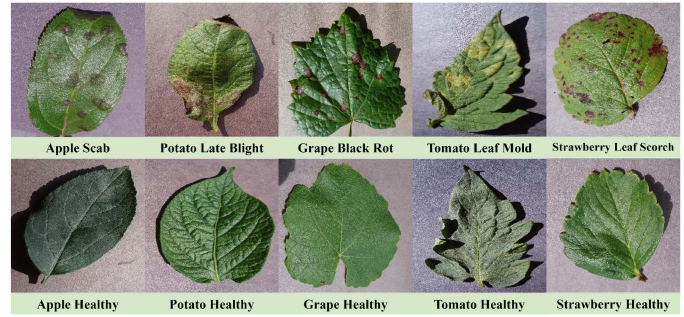


Fig. 1. Some diseased and healthy leaf samples with different characteristics in PlantVillage dataset.

that the proposed model can achieve high accuracy while maintaining a relatively low inference time at 30.46 ms and the least number of parameters at 1.07 million.

- 4) The three contributions mentioned previously possess a certain degree of applicability to address classification problems in other fields. In the extensive testing of the widely used open dataset TrashNet in the field of garbage image classification, the effectiveness of our proposed contributions can still be demonstrated.

II. RELATED WORKS

A. Plant Disease Recognition Based on Deep Learning

Plant disease recognition is an important research topic in the field of machine vision. In the past, traditional plant disease identification was mainly based on traditional image processing methods and manual adjustment of identification schemes to cope with identification tasks in different scenarios [6], [33]. However, these methods need to be used according to the specific situation, and cannot be applied to most natural environments.

Image processing technology based on deep learning has developed rapidly in recent years and has been widely used in plant disease identification tasks [4], [34]. These methods are all based on data-driven machine learning methods, so they are highly dependent on datasets [4], [9]. The public datasets applied to plant disease identification can be divided into multiplant species disease datasets and single-plant species plant disease datasets. The multiplant species disease dataset is considered to be a more suitable dataset for practical agricultural identification [9]. However, in both datasets, single-plant species disease datasets account for the majority, while only two multiplant species datasets are widely used in public [4]. They are PlantVillage dataset [35] and PlantDoc dataset [36]. The PlantVillage dataset is built on controlled environments and contains more than 50 000 images belonging to 38 common plant diseases while PlantDoc dataset has only more than 2000 images belonging to 14 plant diseases. Since the total amount of the PlantDoc dataset is low and it is mainly used for model visualization, the PlantVillage dataset is mainly used for studying deep learning-based plant disease recognition. Fig. 1 indicates some samples in PlantVillage dataset.

The early studies on this dataset mainly focus on comparing different existing deep learning methods. The experiments done by Mohanty et al. [37] on the PlantVillage dataset are early representative studies on this dataset. They compared AlexNet and GoogleNet on this dataset and found that GoogleNet has a more obvious advantage. After that, Too et al. [38] compared the performance of VGG16, Inception V4, DesNets, and other classic network architectures in the plantvillage dataset, and concluded that DesNet 121 can produce the best recognition effect (accuracy rate of 99.75%) while VGG16 can produce the worst recognition effect (accuracy rate of 81.83%). In recent research, some cutting-edge network architectures were included in the comparison. Saleem et al. [43] compared the convolutional neural network (CNN) architectures and optimizers in recent years and finally concluded that the combination of Xception and Adam optimizer can obtain the highest accuracy rate of 99.81% in the PlantVillage dataset. Besides, Atila et al. [44] compared the EfficientNet [45] architecture with other classic deep learning models and combined the transfer learning method to obtain the highest accuracy rate of 99.97% for the B4 structure of EfficientNet (the division ratio of training set and test set is 12:1). However, these general classification models are not specifically designed for identifying plant diseases, which limits their performance when tested on this dataset.

Recently, research has mainly focused on improving training methods, enhancing the CNN architectures, and reconstructing classification networks to address the limitations of existing classification models in training on this dataset. In improving training methods, Liu et al. [9] proposed a weakly supervised training way based on the cluster method and used long short-term memory (LSTM) network to achieve an accuracy rate of 99.78% on the PlantVillage dataset. Besides, in enhancing the CNN architectures, Oyewola et al. [46] enhanced the CNN's ability to handle imbalanced dataset by incorporating shortcut connections and a unique block processing technique. Gao et al. [47] proposed a dual-branch, efficient, channel attention (DECA)-based crop disease recognition model and finally achieved 99.74% accuracy on DECA_ResNet18. Moreover, based on the research of the work of Atila et al. [44], Albattah et al. [48] proposed a network based on DenseNet-77 and introduced the Custom CenterNet framework as a one-stage detector, and this new network finally achieved 99.982% classification results on the PlantVillage dataset. By contrast, the method that Hanh et al. [49] proposed is still based on the EfficientNet architecture and uses pretrained noisy-student weights for this architecture to boost its performance. Ultimately, the method demonstrates that using these weights on the extended and augmented PlantVillage datasets can achieve an average of 99.997% classification accuracy on EfficientNet's B3 and B5 architectures.

In reconstructing classification networks, Mahmoud et al. [39] used DCGAN and PILAE algorithms to synthesize images to solve the problem of unbalanced categories, whereas Nanekaran et al. [40] proposed a hue, saturation, and intensity-based and LAB(L*: Lightness; A*: Red/Green Value; B*: Blue/Yellow Value)-based hybrid segmentation algorithm to

segment images and then classify them for recognition. Moreover, Almadhor et al. [41] utilized the ΔE color difference to isolate diseased areas in guava plants. They extracted feature vectors using color histograms and texture local binary pattern (LBP) features, and ultimately employed a machine learning classifier to identify typical diseases affecting guava plants. On the other hand, some machine learning methods, such as ensemble learning, were also used to build new classification models. Bansal et al. [50] ensembled the predictions of the pretrained DenseNet121, EfficientNetB7, and EfficientNet Noisystudent via averaging to improve the recognition effect of apple leaf diseases. Besides, Khanramaki et al. [19] proposed an ensemble classifier based on deep convolutional neural networks for the identification of three common citrus pests. This classifier is constructed based on the diversity of classifier level, feature level, and data level, and it uses data augmentation, transfer learning, and other methods to further improve the training effect of the model. Finally, this ensemble model can achieve an accuracy of 99.04%, which is better than the recognition accuracy of other mainstream CNN schemes. Vallabhajosyula et al. [18] ensembled different pretrained classic models in groups of three. In the end, 100% recognition accuracy can be obtained on the PlantVillage dataset, and the recognition accuracy of the ensembled models is higher than that of the original submodels. However, although the aforementioned reconstruction methods achieve close to 100% accuracy on the test set, they require significant computational resources as a tradeoff for higher classification accuracy.

B. Accuracy and Inference Time

Even though the advanced network architectures and the improvement methods proposed in recent years have achieved extremely high performance on mainstream plant disease datasets, in the actual recognition environment of plant diseases, the huge model architecture is often difficult to use due to the limitations of equipment performance. In order to facilitate the deployment of the network on low-computing devices, some works in recent years have focused on building miniaturized identification networks. MobileNet [52] proposed a depthwise separable convolution structure, which can greatly reduce the computational complexity of the network. Kc et al. [26] used a deep separable structure to reduce the number of recognition network parameters to one-sixth of MobileNet and still achieved 98.34% accuracy on PlantVillage. Chen et al. [22] constructed MobileNet-Beta by adjusting the end structure of MobileNet-V2 [53] and using the transfer learning method. Finally, they achieved a recognition accuracy of 99.85% on the public dataset. Moreover, they also proposed several novel lightweight recognition models specifically designed for small-scale crop disease datasets [27], [28], [29]. Based on depthwise separable convolution, these models effectively leverage the favorable characteristics of classic network architectures, such as SE block and inception module, to achieve high-precision disease recognition with a low parameter count.

On the contrary, some studies have proposed that compared with other network structures, deep learning models with low computational load or a low number of parameters may not necessarily have faster inference speed on low-performance devices [30], [31], [54]. Therefore, compared with conventional experiments, conducting experiments on specific low-performance devices can provide a more accurate representation of the model's performance in low-performance scenarios. Gonzalez-Huitron et al. [25] selected a subset of tomato leaves on PlantVillage as the research object and conducted experiments on Raspberry Pi 4. They used transfer learning and data augmentation in training and compared the training results of seven advanced networks. In addition, Mora et al. [55] used transfer learning to train five advanced networks in the PlantVillage dataset. They tested not only the accuracy of these networks but also their speed in the Raspberry Pi4. Finally, the test results indicate that VGG16 is the network with the highest accuracy while MobileNetV2 is the network with the fastest inference speed among the five networks.

Plant disease recognition relies mostly on low-power devices with high demands for both accuracy and speed. Based on the analysis of existing work, this article proposes improvements in network architecture and training methods, as well as a new lightweight ensemble learning approach, to achieve a high-accuracy real-time recognition model in low-power scenarios.

III. MATERIALS AND METHODS

In this section, we describe the details of the scheme for building a high-precision real-time recognition network and provide relevant theoretical basis. The content mainly includes dataset preprocessing, model structure, model training strategies, and a lightweight ensemble learning method.

A. Dataset Preprocessing

To compare the training performance of other models, we did not augment the original dataset before training started. In the data preprocessing stage, two partitioning methods were utilized for the PlantVillage dataset. The first method divided the training and test sets in an 8:2 ratio for fivefold cross-validation, whereas the second method partitioned the dataset into training, validation, and test sets in an 8:1:1 ratio to facilitate comparison with other techniques. In addition, online enhancement of images was applied during training to reduce overfitting during training. Specifically, the online enhancement process comprises randomly triggering horizontal flipping of the image, vertical flipping of the image, changing the brightness, contrast and saturation of the image, and cutting out part of the image and scaling it to a specified size. Meanwhile, some offline enhancement methods were incorporated into ensemble learning to produce more diversified iterative outcomes. Fig. 2 outlines the specific execution steps of these methods, while minor adjustments made to these methods during various processes will be elaborated in later chapters.

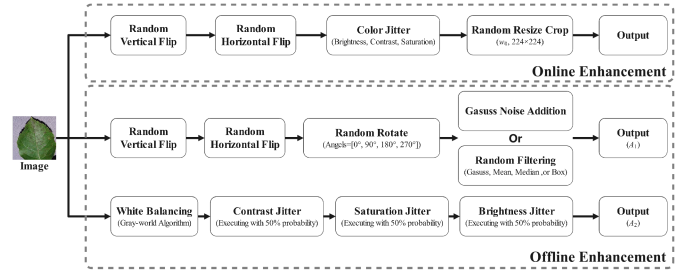


Fig. 2. Flowchart of image preprocessing. In this chart, the online enhancement is used in all training processes, while offline enhancement is only used in the ensemble learning method.

B. CNN Basic Optimization Strategy

In order to build a plant disease identification algorithm that can recognize real-time with high accuracy, the Roofline theory [30] is used by us to measure the inference performance of the network. The model defines the upper limit of the computing strength of the hardware by combining the computing power π (FLOPS/s) and bandwidth β (Byte/s) of the target hardware device

$$I_{\max} = \frac{\pi}{\beta}. \quad (1)$$

The Roofline model provides a way to calculate the computational intensity T of the network model based on the amount of computation (time complexity C) of the deep learning model and the amount of memory access (space complexity O)

$$T = \frac{C}{O}. \quad (2)$$

The limitations of hardware performance on AI models are usually presented in the form of a 2-D axis, where the X-axis represents the operational intensity, i.e., the amount of operations per unit memory access, and the Y-axis represents the attainable performance. Moreover, the hardware performance is reflected in the performance line composed of two parts: slant line and horizontal line, where the slant line represents the upper limit of memory bandwidth, and the horizontal line represents the upper limit of computing performance [56]. The closer the performance of the deep learning model is to the contact point of these two lines in this figure, the more the model fully utilizes the performance of the hardware device.

In order to create a plant disease recognition model that is applicable to Jetson Nano, we utilized the Roofline theory to construct a hardware performance curve specific to Jetson Nano [32] and compared multiple classical deep learning methods. Based on Fig. 3, after excluding classical deep learning methods with slow inference speed, SqueezeNet, GoogLeNet, and ShuffleNetV2 are closer to the optimal operating point compared with other classical deep learning methods. In addition, these networks are characterized by having a small number of convolutional layers. Previous studies have shown that parallel structures can reduce the number of layers in the network, but excessive parallel structures can increase model fragmentation and significantly reduce model inference speed [31].

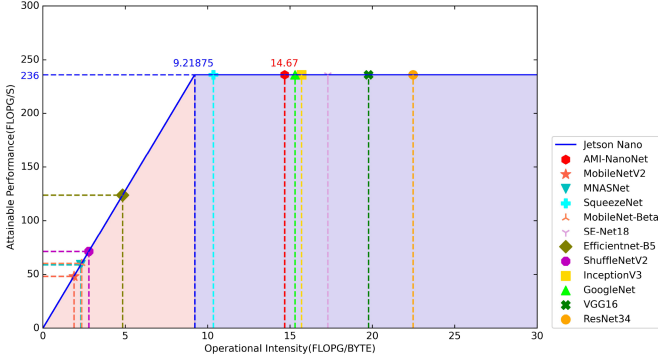


Fig. 3. Computational efficiency comparison of deep learning frameworks based on Roofline model for Jetson Nano. The area covered by the horizontal blue line is compute bound, indicating that the amount of computation is the main factor affecting the model inference speed. Besides, the area covered by the slant line is memory bound, indicating that the amount of memory access is the main factor affecting the model inference speed. The data types for this analysis are all based on FP32.

Combining these viewpoints, we propose an idea to construct the lightweight model suitable for Jetson Nano: to design an efficient parallel architecture based on Roofline model, which will be used in the model as little as possible.

C. Framework of AMI-NanoNet

Since the inverted residual block can greatly reduce the amount of computation and achieve relatively excellent results in related experiments on plant disease datasets [25], [52], [53], [55], the backbone architecture of our network is constructed based on this block. Besides, depthwise separable convolutions are limited to a certain number to reduce excessive memory accesses [31].

In recent research on plant disease recognition, the network based on Inception architecture has high recognition accuracy at a relatively shallow network depth [9], [42], [43], [44], while the network architecture based on depth-separable convolution has relatively high recognition accuracy with low floating point of operations (FLOPs) [25], [52], [53], [55]. According to this phenomenon, we build AMI module based on the InceptionV3-D and depthwise separable convolutions. Notably, AMI (V3-D) means combining this inception module with the attention mechanism. This idea is based on the use of efficient feature extraction mechanisms of different types within parallel structures. Unlike convolutional block attention module (CBAM) [57], in order to reduce the fragmentation of the convolutional network, this module is divided into two parts, A and B. For part A, the two parallel structures of the InceptionV3-D module are replaced by the extreme inception module [58] and the spatial attention module, respectively. The rationale for this replacement is twofold: to enhance the density of information acquired by the front end of the network and to reduce the loss when the feature map size shrinks. For part B, the addition of SE module is mainly used to reduce the loss caused by the reduction of network dimension. The validation of the two modules is described in Section IV-B.

Algorithm 1: Auxiliary Training Strategy.

Input: The network NET to be trained, an online enhancement method M , cropping ratio $\omega \in [\omega_0, 1]$, a set of iterations for which the cropping ratio is adjusted $\mathbf{u}$, a training method S , the number of errors e , the expected accuracy α , #iterations N

Output: NET

All parameters are initialized;

Set e equal to 0;

for $Epoch$ in $range(N)$ **do**

if $Epoch \in \mathbf{u}$ **then**

 Reduce the ω_0 ;

$M \leftarrow M_\omega$;

end

for i in $range(n)$ **do**

$NET \leftarrow S(M(\chi_i), NET)$;

end

for i in $range(l)$ **do**

if $NET(\chi'_i) \neq \gamma'_i$ **then**

$e = e + 1$;

end

end

if $1 - \frac{e}{l} \geq \alpha$ **then**

 Stop Training;

end

end

Based on our proposed modules and the inverted residual block, the feature extraction part of the proposed network only consists of six modules and one ordinary convolutional layer. The end of this part is followed by a global pooling structure and a fully connected (FC) layer. Based on this network developed for low-power devices, such as Jetson Nano, and using the AMI structure, we call it AMI-NanoNet. An overview of this model is presented in Fig. 4, and a Roofline model-based computational performance analysis of this model and other models is presented in Fig. 3. In this analysis, AMI-NanoNet is at compute bound and its computing utilization efficiency is only lower than SqueezeNet. Moreover, in compute bound, AMI-NanoNet has the least amount of computation. By combining the computing power of the hardware device with the calculation amount of the model, the time t for the model to calculate a single image can be roughly estimated

$$t = \begin{cases} \frac{C}{\pi}, & \text{in Compute Bound} \\ \frac{Q}{\beta}, & \text{in Memory Bound.} \end{cases} \quad (3)$$

Model performance test based on this theoretical estimation method is presented in Section IV-D.

D. Feature-Guided Curriculum Learning

Curriculum learning was proposed as an auxiliary strategy to enhance the generalization ability of machine learning training

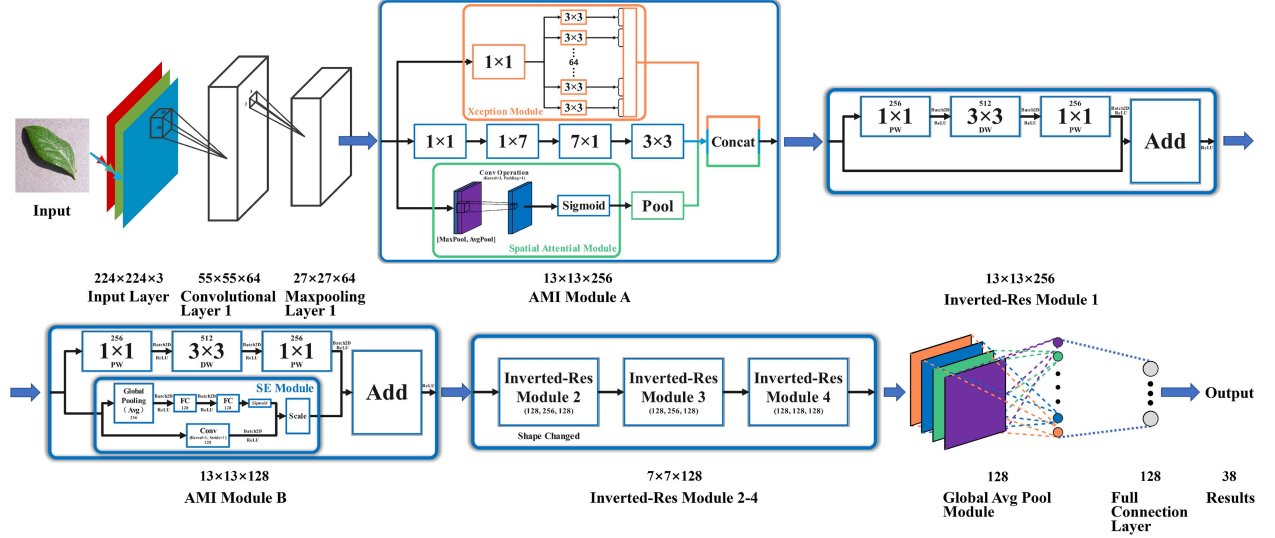


Fig. 4. Proposed architecture: AMI-NanoNet.

strategies and accelerate the speed of convergence [58], [59], [60]. We propose the FGCL strategy to assist training mainly for two purposes. First, this auxiliary method can improve the generalization ability of the network and reduce the overfitting of training. Second, training the network on images of varying scales and distributions can improve its stability during the subsequent ensemble learning step.

In our strategy, the level of training difficulty is associated with the dissimilarity between the features input to network and the features of the original images in dataset. When there is a greater deviation between the input features and the features of the original images, we consider such input features to be more challenging for the network to learn. Consequently, we can amplify the intensity of online augmentation to increase the deviation of the input features from the features of the original images. In a word, the FGCL strategy is to adopt online data augmentation methods with different intensities in different training stages, and this intensity changes from weak to strong with the progress of training to form the process of training from simple to complex.

In this strategy, we adjust the random cropping range of the input image to control the ease with which the data input to the network is learned. This cropping method selects the size of the cropped area according to the ratio ω and performs it at random positions in the input image. Then, the cropped image is reconstructed into a 224×224 size image by bilinear interpolation. During this online enhancement process, ω is randomly set to a value in the range from ω_0 to 1. Therefore, reducing ω_0 is equivalent to increasing the difficulty of the sample being learned by the model. Specifically, for the convenience of illustration, we name the class label set, training set, the test set, and the set of iterations for which the cropping ratio is adjusted as $\mathbf{c}$, $\mathbf{D}$, $\mathbf{D}'$, and $\mathbf{u}$, respectively, where $\mathbf{c} = \{c_1, c_2, \dots, c_R\}$, $\mathbf{D} = \{(\chi_1, \gamma_1), (\chi_2, \gamma_2), \dots, (\chi_n, \gamma_n) | \chi_i \in \mathbb{R}^{m \times m}, \gamma_i \in \mathbf{c}\}$, $\mathbf{D}' = \{(\chi'_1, \gamma'_1), (\chi'_2, \gamma'_2), \dots, (\chi'_l, \gamma'_l) | \chi'_j \in \mathbb{R}^{m \times m}, \gamma'_j \in \mathbf{c}\}$,

and $\mathbf{u} = \{u^{(1)}, u^{(2)}, \dots, u^{(k)}\}$. In the case of ignoring batch size, this strategy can be described in detail in Algorithm 1. What is more, experiments on the effectiveness of this strategy are presented in detail in Section IV-B.

E. Category-Based Ensemble Learning Method

Due to the randomness of the training method of a single learner, there is some uncertainty in the improvement of recognition accuracy. Ensemble learning accomplishes learning tasks by building and combining multiple learners, and the improvement of accuracy in this way can be built on the basis of a single learner. Nevertheless, recent approaches to applying ensemble learning models in the field of plant disease identification use entire deep learning models for ensemble [18], [19], [50], [51]. This ensemble method is not suitable for low-power devices due to excessive computational complexity.

On the other hand, the ensemble learning methods applied to deep learning models are mainly divided into bagging and boosting. The bagging method improves the recognition effect by enhancing the difference in the learning effect of each learner, and the boosting method sequentially adjusts the distribution of samples to train multiple learners to improve the recognition effect. Since bagging can be trained in parallel, it is more widely used in deep learning than boosting.

In order to minimize the computational increase caused by the ensemble learning method, we only ensemble the FC layers of the network. It means that the composition of one learner solely comprises the structure connecting 128 neurons derived from global pooling to 38 output neurons. Moreover, the feature extraction layers before the FC layers and the FC layers within the original network are not retrained in the ensemble learning stage. In this way, the FC layers connected to feature extraction layers can be considered as a new learner. We use the symbol A_0 to denote the initial learner, and the symbols A_1 and A_2 to denote the newly constructed learners. These learners are connected to feature extraction layers with global average pooling in the same way, and the final result

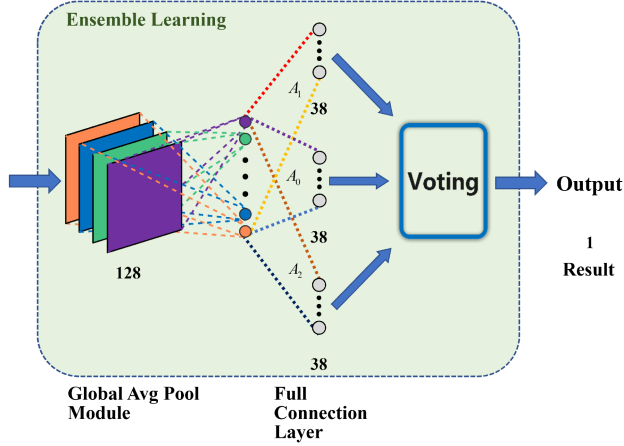


Fig. 5. Proposed ensemble method for lightweight models. In this way, only A_1 and A_2 need to be retrained to enhance the difference between models. The final ensemble result depends on how the three models vote according to different weights. The sum of the weights of A_1 and A_2 should be greater than the weight of A_0 , and the weight of A_0 should be greater than the weight of any other single learner.

is selected by voting according to the weights of different proportions. This ensemble method is illustrated in detail in Fig. 5. For the convenience of illustration, we represent the output value of the learner on sample χ as an R -dimensional vector $\mathbf{H}_b(\chi) = (\text{NET}_{A_b}^1(\chi), \text{NET}_{A_b}^2(\chi), \dots, \text{NET}_{A_b}^R(\chi))$. In this vector, $\mathbf{H}_b^i(\chi)$ is the output value of the learner A_b on the class label c_j . Based on our hard voting criterion, class probabilities need to be converted to class labels. In order to avoid sequence errors during the conversion process, the class probabilities need to be converted to the range 0–1 first

$$\mathbf{G}_b(\chi) = \text{sigmoid}(\mathbf{H}_b(\chi)). \quad (4)$$

Then, we turn the largest probability to 1 and the rest to 0

$$\mathbf{F}_b = \mathbf{1}_{i=\arg \max_i \mathbf{G}_b(i)}. \quad (5)$$

Ultimately, the result of voting can be expressed as V

$$V = \arg \max_b \sum_i w_b \mathbf{F}_b(i), w_0 > w_1 + w_2 \text{ and } w_1 = w_2. \quad (6)$$

This voting method can ensure that the identification accuracy after ensemble is inherited from the model before ensemble. Therefore, the fewer the number of learners, the better the ensemble performance of this method will be.

Due to the computational complexity limitations, the conventional boosting method is not applicable for our model. In contrast, we propose a new training method based on the bagging method to train the ensemble learning model.

The normal bagging method uses the bootstrap to enforce training differences between ensemble models. However, this approach generally requires a large number of learners. Because bootstrap can lead to a significant drop in the accuracy of a single learner, and insufficient number of learners makes it difficult to reflect the improvement in accuracy brought about by this training difference in the ensemble results. Our proposed

iterative approach aims to address these challenges by using different ways to modify the distribution of the training set and setting specific iteration conditions, thereby generating individual learners with low error rates and high diversity.

Specifically, this method divides the dataset according to the two levels in Fig. 6 and uses offline enhancement methods for learners of A_1 and A_2 to enhance the incorrect images in their respective training sets. Such enhancements include horizontal and vertical flipping, rotation, color equalization, contrast change, noise addition and filtering. In order to limit the degree of image variation, the random range of these variations is limited to a low level and is mostly used in each enhancement based on a 50% probability. On the other hand, to accumulate diversity at each iteration and further increase the differences between the two learners, we applied morphology-based and content-changing enhancement methods to A_1 , while the color-based enhancement methods were used for A_2 . In Fig. 2, the detailed process of offline learning is shown.

Among them, the images that the A_1 learner judges incorrectly in the Level 1 category will be enhanced in its training set after each training reaches the specified accuracy α' , while the incorrect images judged by A_2 learner in the Level 2 category will be enhanced in its training set after each training reaches the α' . If the desired accuracy α' is not achieved within the max epochs of algorithm iterations, offline enhancement will not be performed and training will be restarted with modified random seeds. Moreover, to ensure the model's adequate learning on the dataset, we set a base number of iterations N' such that the number of epochs per training session exceeds this value. The reason we opted not to copy the images but to enhance the images is to increase the richness of the modified training set and the difference between the two learners after one training session. Using this method can increase the number of iterations of our algorithm and decrease the average error rate of the two learners, while using direct replication was observed to be difficult to make the algorithm more than two iterations in our preexperiments.

Finally, the minimum range within which the images can be cropped will be increased and the range of image color jitter will be reduced to reduce the variation range of the input features in the next training round. Besides, if the trained learners do not improve accuracy after ensemble, these learners will be retrained based on their respective enhanced training sets. Based on these methods, the detailed training algorithm is indicated in Fig. 7.

IV. EXPERIMENTS AND RESULTS

In this section, we conduct a series of experiments to verify the effectiveness and advancement of our proposed methods. First, we introduce the equipment and related preparations for our experiments. Second, we use ablation experiment to test the effectiveness of our proposed methods for improving network accuracy. This experiment includes two parts, namely, the network framework test and the learning algorithm test. Next, we examine the effectiveness of ensemble learning by combining the accuracy with the differences identified by different learners.

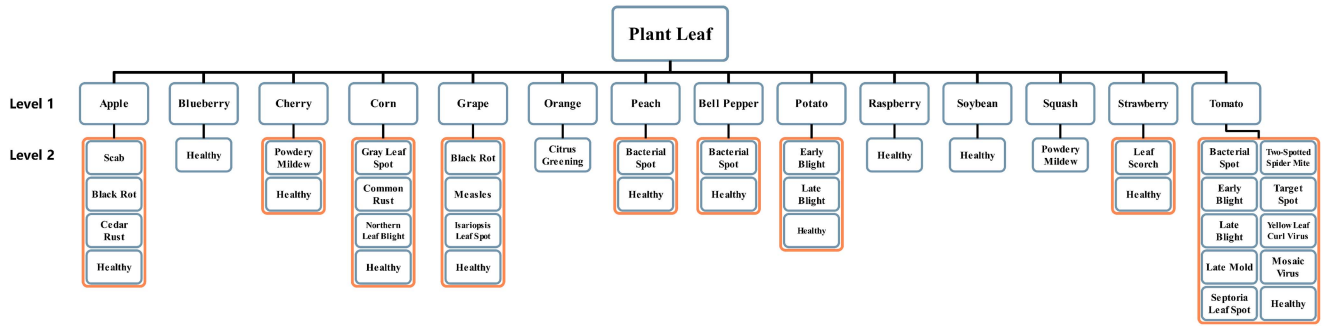


Fig. 6. Schematic diagram of the category division of the PlantVillage dataset. In this diagram, Level 1 represents the classification of plant leaf species, and Level 2 represents the classification of plant leaf disease categories.

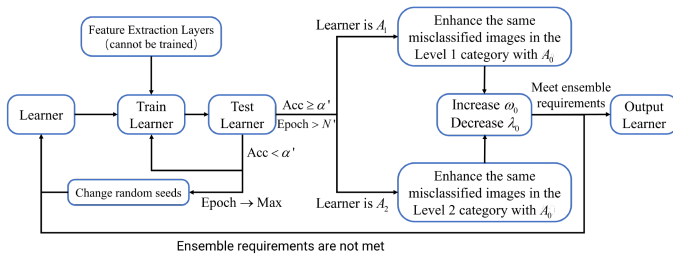


Fig. 7. Schematic diagram of the training algorithm of the learners. For the convenience of expression, we name the base number of iterations, target accuracy, and image color jitter rate (including contrast, brightness, and saturation) as N' , α' , and $\lambda \in [1 - \lambda_0, 1 + \lambda_0]$, respectively.

TABLE I
HARDWARE AND SOFTWARE ENVIRONMENT

Configuration Item	Algorithm Server for Training	Jetson Nano
System	Ubuntu 18.04.5 LTS	Ubuntu 18.04 LTS
CPU	Intel(R) Core(TM) i9-10920X CPU @ 3.50GHz	ARM A57 @ 1.43GHz
GPU	NVIDIA GeForce RTX 3090 (24G)	NVIDIA Maxwell @ 921MHz
RAM	64G	4G
CUDA	11.1	10.0
Cudnn	8.0.3	7.6.3
Pytorch	1.10.1	1.4
Torchvision	0.11.2	0.5
Python	3.8.5	3.6

Finally, we compare our proposed methods with other state-of-the-art methods to illustrate how advanced our methods are. In addition, we also test the applicability of these methods to other domain datasets.

A. Experimental Setting

Our experiments are mainly performed on two platforms. One is our team’s algorithm server, and the other is the Jetson Nano platform for testing. Table I describes these device information and configuration environment in detail.

Besides, the training of our model is based on the algorithm server. Since the training difficulty of AMI-NanoNet is different

TABLE II
TRAINING ENVIRONMENT

Configuration Item	AMI-NanoNet	Learners
Batch Size	256	256
Optimizer	Adam	Adam
Criterion	CrossEntropyLoss	CrossEntropyLoss
Scheduler	OneCycleLR	OneCycleLR
Max Learning Rate	0.0065	0.0015
Steps Per Epoch	1	1
Pct_start	0.24	0.4
Diversity Factor	5.5	$5.5(A_1)/5(A_2)$
Final Diversity Factor	4000	1000
Max Epoch	600	150
ω_0	0.07	0.065
λ_0	0.16	0.24
N'	-	20
Enhanced Images Per Error	-	12

from that of the learners, there are certain differences in the related training hyperparameters. Table II indicates different hyperparameter settings for the two types of training. In this table, the hyperparameters max learning rate, steps per epoch, Pct_start, diversity factor, and final diversity factor are included as auxiliary parameters of the Scheduler (OneCycleLR) to adjust the curve generated by the learning rate, which varies with increasing epochs. Furthermore, we did not perform pretraining, such as transfer learning, etc., before model training to demonstrate the effectiveness of our method. The initialization of model parameters is performed according to the default initialization methods of the relevant Pytorch version.

To evaluate the classification performance of our methods, the macroaveraging F1 score (MF1) and accuracy (Acc) are chosen as our evaluation metrics.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (7)$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (8)$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (9)$$

TABLE III
MODULE REPLACEMENT INSTRUCTIONS

Model Name	Replaced Module	New Module
AMI-NanoNet	-	-
AMI-NanoNet-A	Spatial Attentional Module in AMI-A	Initial Module in InceptionV3-D
AMI-NanoNet-B	Xception Module in AMI-A	Initial Module in InceptionV3-D
AMI-NanoNet-C	AMI Module A	InceptionV3-D
AMI-NanoNet-D	AMI Module B	Inverted Residual Block
AMI-NanoNet-E	AMI Module A&B	InceptionV3-D&Inverted Residual Block
AMI-NanoNet-F	AMI Module A	Inverted Residual Block
AMI-NanoNet-G	AMI Module A&B	Inverted Residual Block

$$MF1 = \frac{\sum_{c=1}^R F1_c}{R} \quad (10)$$

$$Acc = \frac{TP + TN}{TP + FP + TN + FN} \quad (11)$$

where TP = true positive, FP = false positive, TN = true negative, FN = false negative, $F1_c$ is per-class F1-score of class c , and R is the number of categories in the test set [25], [61]. Among these two evaluation metrics, MF1 can complement the shortcomings of the Acc metric when dealing with the problem of category imbalance.

B. Ablation Study

In order to test the improvement effect of AMI Module in the network framework and to test the improvement of our proposed model training method compared with the traditional training methods, we designed relevant ablation experiments for testing.

First, we need to set the hyperparameters required for this experiment. According to the conclusion of literature [43], the Adam optimizer showed better performance than other commonly used optimizers in the test of the PlantVillage dataset. Furthermore, we tested various hyperparameter combinations before performing the ablation study. According to the fivefold cross-validation results of AMI-NanoNet on the PlantVillage dataset, the hyperparameter combination shown in Table II is the optimal combination among the various hyperparameter combinations we tried.

Second, to validate the effectiveness of the AMI module, we separately restore each modified component of AMI modules A and B, relative to the original structure, to assess the efficacy of the modifications. Furthermore, we substitute the inverted residual block for AMI modules A and B, respectively, to evaluate the effectiveness of the two modules within the network. Table III tabulates the specific structures we replaced.

We employ fivefold cross-validation to compare the performance of these models. The dataset is partitioned into five subsets, with one subset being designated as the test set in each iteration while the remaining four serve as the training set. The results of the relevant tests are presented in Fig. 8. Due to the small difference in the calculation amount of modules before and after replacement, the theoretical inference time of these models is concentrated at 1.28 and 1.48 ms. In addition, it is

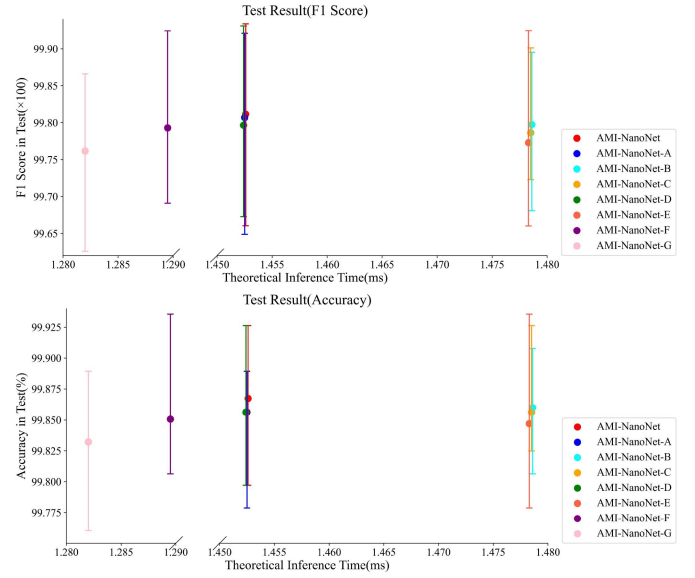


Fig. 8. Results of ablation experiment conducted on the AMI module.

TABLE IV
CROP RANGE ADJUSTMENT

Epoch(u)	Adjustment Ratio of ω_0
25	0.50
75	0.50
150	0.70
250	0.80

clear that the original structure of AMI-NanoNet can achieve the highest average accuracy and average MF1, even though its theoretical inference time is not the least. On the other hand, by comparing the different substitution results, we can summarize the following conclusions. First, the results of C and F indicate that the parallel structure exhibits better performance compared with the inverted residual block, despite a slight increase in model inference time. Second, the difference results of A, B, and C show that the modifications to AMI Module A (AMI-A), in contrast to InceptionV3-D, result in better performance and lower inference time. Third, D, F, and G show the effectiveness of the AMI modules A and B. Moreover, AMI module A has a greater impact on network performance than the AMI module B, and the joint use of AMI module A&B is better than the inverted block in our network.

Third, we conducted the fivefold cross-validation on our proposed training method (FGCL) and the traditional training method to compare their differences. In experiment with our proposed training method, ω_0 is tuned down from 0.5 to 0.07 with increasing the epoch. However, since each adjustment causes the training curve to oscillate, such adjustments cannot be made continuously and the adjustment amplitude should not be too large. We finally selected the scheme in Table IV for adjustment.

In this way, the trigger interval of the adjustments increases with the epoch, and the magnitude of this adjustment is gradually decreased. By tuning in this way, the oscillation of network training becomes smaller and the occurrence of overfitting is

TABLE V
ABLATION STUDY RESULTS OF TRAINING METHOD IN PLANTVILLAGE

Test Metrics	Proposed Method	Traditional Method
Average MF1($\times 100$)	99.8290	99.8115
Highest MF1($\times 100$)	99.9260	99.9338
Minimum MF1($\times 100$)	99.7160	99.6604
Average Accuracy(%)	99.8710	99.8673
Highest Accuracy(%)	99.9445	99.9263
Minimum Accuracy(%)	99.8064	99.7972

The bold values represent average references.

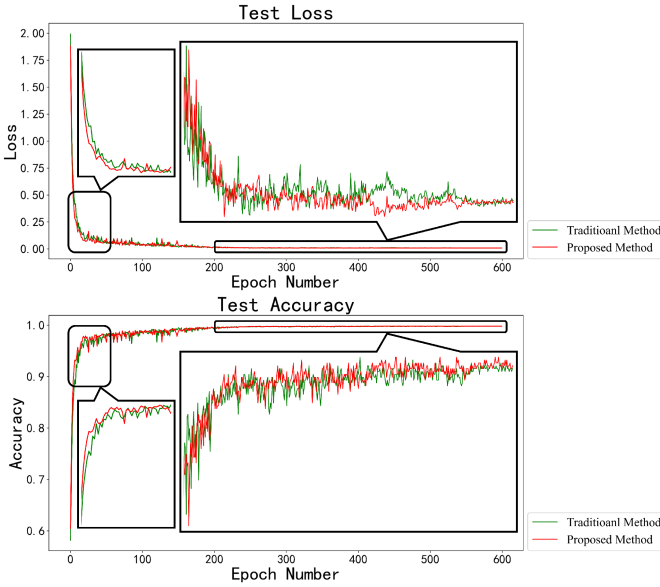


Fig. 9. Training comparison figure of the two methods. In this figure, curves from some stages are magnified to demonstrate the discrepancies between the two methods.

further suppressed. By contrast, in the traditional method, ω_0 remains at 0.07 and will not be changed. Table V tabulates the experimental results in detail.

It is clear that our proposed training method can achieve higher and more stable MF1 and accuracy than the traditional method. In order to further demonstrate the excellent convergence characteristics of this method, we select the training process when the highest accuracy of the two training methods is the same in the fivefold cross-validation for comparison. Fig. 9 shows this comparison in detail. Due to the lower initial training difficulty, our method converges faster than traditional methods in the initial training stage. At the end of training, due to the accumulation of previous training, our method can make the convergence more stable and achieve higher accuracy than the traditional method. Moreover, from this figure, there is a large randomness in the achievement of the highest accuracy of the traditional method, while the highest accuracy achieved by our proposed method can be more easily reproduced.

C. Effectiveness of Ensemble Learning

In bagging-based ensemble learning methods, the difference between classifiers is one of the most important factors affecting

the ensemble effect. As a common statistic, the kappa statistic κ used to measure paired classifiers is widely used to measure the difference of multiple classifiers [62]. By measuring the κ statistics and the average errors between pairs of learners after each iteration, we can test whether the iterative method based on Fig. 7 plays a significant role in improving the accuracy of our network.

Given two classifiers A_a and A_b , and the test set D' , which has l samples. The κ statistic can be defined in

$$\kappa = \frac{p_1 - p_2}{1 - p_2} \quad (12)$$

where p_1 is the probability that two classifiers agree, and p_2 is the probability that two classifiers agree by chance. For the convenience of discussion, we use $d_{\mu\eta}$ to denote the number of test samples for which $\arg \max_R \mathbf{H}_a(\mathbf{x}'_j) = \mu$ and $\arg \max_R \mathbf{H}_b(\mathbf{x}'_j) = \eta$. Through the description of these symbols, the calculation process of p_1 and p_2 can be shown in the following [62]:

$$p_1 = \sum_{\mu} \frac{d_{\mu\mu}}{l} \quad (13)$$

$$p_2 = \sum_{\mu=1}^R \left(\sum_{\eta=1}^R \frac{d_{\mu\eta}}{l} \cdot \sum_{\eta} \frac{d_{\eta\mu}}{l} \right). \quad (14)$$

To describe the average error, we express the accuracy of the b th learner on the test set in the form (11). Then, the average error between a pair of learners can be expressed as

$$\text{Mean Error}_{ab} = \frac{2 - \text{Acc}_a - \text{Acc}_b}{2} \quad (15)$$

where $a < b$ and $a, b \in \{0, 1, 2\}$. To assess the overall difference and mean error among these learners, we use the average method to represent the κ statistic and mean error across all learners

$$\text{Mean Error} = \frac{\text{Mean Error}_{01} + \text{Mean Error}_{02} + \text{Mean Error}_{12}}{3} \quad (16)$$

$$\kappa = \frac{\kappa_{01} + \kappa_{02} + \kappa_{12}}{3}. \quad (17)$$

To enable comparison with similar methods, we randomly partition the dataset into training, validation, and test sets in an 8:1:1 ratio. The hyperparameters proposed in Table II and the FGCL method are utilized to retrain AMI-NanoNet, and the model with the highest validation set accuracy is selected for the subsequent phase of the experimentation. It should be noted that if there are multiple maximum values on the validation set, we choose the model with the smallest corresponding loss.

Then, we obtain a model with a validation set accuracy of 99.8708%, which achieve a test set accuracy of 99.8343%. In order to further improve the highest accuracy that our model can achieve, this model is used for further ensemble learning training. By using (12) and (15), the results of testing these learners trained by the algorithm in Fig. 7 on the test set can be depicted in Fig. 10. In this process, due to the low number of these learners, we set $\alpha' = 99.8708\% - 0.02\%$ to control the quality of each learner. To mitigate the issue of overfitting

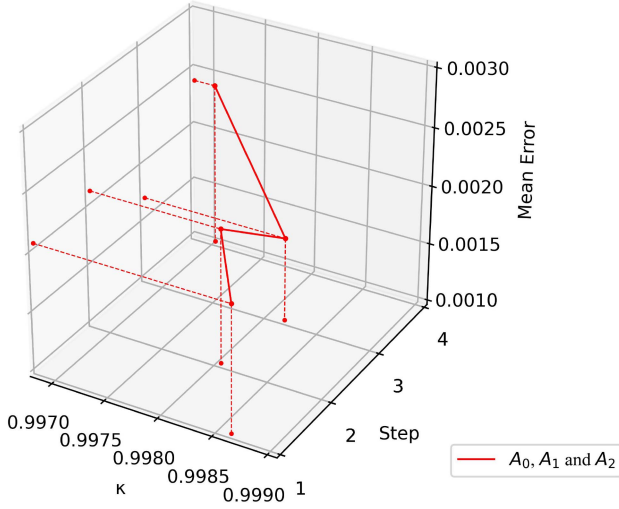


Fig. 10. Variation of the difference among these learners in iterations by using the κ statistic and mean error.

caused by training these learners solely, we slightly increase the complexity of online learning. Moreover, we adjust the training parameters for both learners due to differing levels of difficulty in training on two distinct training set distributions. The complete list of these parameters is available in Table II.

Since the model accuracy before ensemble is close to 100%, the number of times that the learner is successfully iterated is only four times. However, it can be clearly seen from Fig. 10 that as the iteration progresses, the scatter positions of each pair of learners in this figure are continuously shifted to the left. This proves that the differences between learners are continuously enlarged as the iteration progresses. Finally, our ensemble learning method achieves an accuracy of 99.8708% on the test set, while it achieves an accuracy of 99.8892% on the validation set. Moreover, the change in the distribution range of training set samples increases the difficulty of training the ensemble learning model. Consequently, the mean error of the learners increases with each iteration, which primarily accounts for the relatively low number of iterations. However, even if the identification accuracy of an individual learner is significantly lower than that of the original model, the final ensemble result still shows remarkable improvement due to the increased identification difference between the two learners. Specifically, the accuracy of A_1 learner increases by 0.11%, that of A_2 learner by 0.22%, and that of the original model by 0.04%.

Since the impact of the newly added ensemble process on the theoretical inference time of the entire model is less than 0.002 ms and the ratio of this value to the total inference time is less than 0.14%, we can ignore the computational time loss caused by this process.

D. Comparison With Other Architectures

Referring to the hardware and software settings in Table I, we compare the performance data of our network (including ensemble learning) with other advanced networks based on

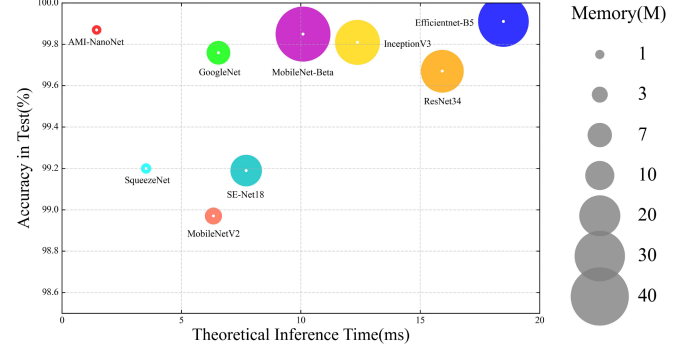


Fig. 11. Performance comparison of AMI-NanoNet and the state-of-the-art networks. The accuracy data in this figure are from Liu et al. (SqueezeNet, GoogleNet and ResNet34) [9], Atila et al. (InceptionV3 and Efficientnet-B5) [44], Gao et al. (SE-Net18) [47], and Chen et al. (MobileNetV2 and MobileNet-Beta) [22]. Moreover, these models were all trained on a training set sampled at an 80% ratio from PlantVillage.

theoretical inference time and actual inference time on the server and Jetson Nano, respectively. In this comparative experiment, the classification performance of the advanced models being compared is based on publicly reported results. In addition, these models were all trained on a training set sampled at an 80% ratio from PlantVillage. Besides, the state-of-the-art networks being compared are selected based on our requirement. This requirement is that the theoretical inference speed of the model is less than 20 ms and the identification accuracy is higher than 98.5%. We calculate the approximate theoretical inference time of these models according to the method given by (3), and the specific experimental data are depicted in Fig. 11.

As can be seen from the figure, our model has the least amount of parameters at 1.07 million and has the fastest inference speed at 1.45 ms based on our theory. Moreover, in terms of identification accuracy, only Efficientnet-B5 of these models surpasses the recognition ability of our model. However, the theoretical inference time of this network is too long and contains too many parameters to make it unsuitable for real-time recognition of low-power devices.

Nevertheless, due to the hard-to-quantify differences in algorithm implementation, operating environment, code writing, etc., the model inference time we estimate may not be achievable in actual testing. Therefore, we need to run these models on the actual Jetson Nano platform to test their real inference time. Our experiment is based on the platform environment shown in Table I, and the test results (actual inference time) are obtained based on the average of 10 000 tests. The detailed results of this experiment are detailed in Fig. 12.

In this experiment, the actual inference speed of these models is slower than the theoretical inference speed due to differences in the platform on which the algorithm is implemented and other factors that are not quantifiable. By comparing the differences between SE-Net18, MobileNetV2, MobileNet-Beta, and ResNet34 in the two experiments, the Pytorch framework running on the Jetson Nano platform may have some defects in the algorithm implementation of depthwise separable convolution. As a result, MobileNetV2, Mobilenet-Beta, and other networks

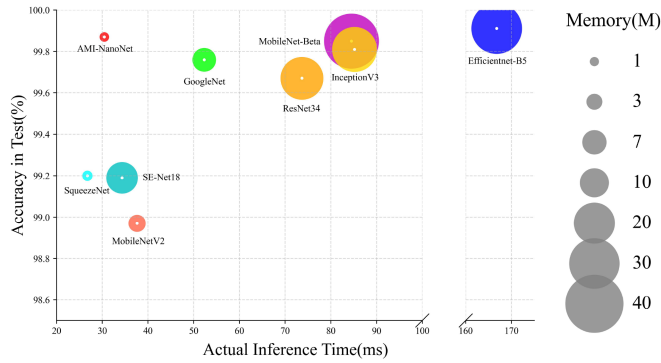


Fig. 12. Performance comparison of models based on the Jetson Nano platform. The sources of accuracy data for these models are the same as in Fig. 11.



Fig. 13. Some samples with different characteristics in the TrashNet dataset and the PlantVillage dataset. The samples in the TrashNet dataset were obtained from natural environments, whereas the samples in the PlantVillage dataset were acquired from controlled environments.

that use a large number of deeply separable convolutions are run on Jetson Nano at a low inference speed. However, the inference speed (30.46 ms) of our network is second only to SqueezeNet (26.76 ms) under this experimental condition, and it still has advantages over other advanced networks in terms of parameter quantity and recognition accuracy. This means that our proposed method is more suitable than other state-of-the-art methods for efficient real-time identification of plant diseases on a low-power platform, such as the Jetson Nano.

E. Applicability Test

Identifying plant diseases in real environments often faces variable external factors. These factors are not necessarily covered by existing datasets. Therefore, in order to test whether our proposed method can be applied to a wider range of scenarios, we selected a nonplant disease classification dataset for testing. Since PlantVillage is a dataset with a large number of categories, a large amount of images, and a controlled environment, we specifically select a dataset with only a small number of categories, a small number of images, and a natural environment. This dataset is TrashNet with 2527 images and six categories, which are cardboard (403), glass (501), metal (410), paper (594), plastic (482), and trash (137) [63]. Fig. 13 illustrates some samples from the two datasets. It is clear that the backgrounds of these images from the PlantVillage dataset exhibit a higher degree of uniformity compared with those from the TrashNet dataset.

TABLE VI
HYPERPARAMETERS MODIFICATION

Configuration Item	AMI-NanoNet	Learners
Max Learning Rate	0.005	0.003
Diversity Factor	-	$6.5(A_1)/7(A_2)$
ω_0	-	0.068
Enhanced Images Per Error	-	$4(A_1)/2(A_2)$

TABLE VII
ABLATION STUDY RESULTS OF TRAINING METHOD IN TRASHNET

Test Metrics	Proposed Method	Traditional Method
Average MF1($\times 100$)	95.1229	94.8277
Highest MF1($\times 100$)	97.8952	97.5413
Minimum MF1($\times 100$)	92.4488	92.7220
Average Accuracy(%)	95.3785	94.9800
Highest Accuracy(%)	97.6096	97.2112
Minimum Accuracy(%)	93.2271	92.8287

The bold values represent average references.

First, we need to validate the effectiveness of FGCL on this dataset. Based on the method proposed in Section IV-B, we modified some of the training hyperparameters to enable AMI-NanoNet to be trained to a higher accuracy on this dataset. The changes to the hyperparameters are detailed in Table VI. Due to the small size of the TrashNet dataset with only a few samples, we employ tenfold cross-validation in our experiments to ensure the reliability of the results.

Table VII tabulates the results of the tenfold cross-validation. It is evident from MF1 and accuracy measures that our proposed method still outperforms traditional methods on this dataset.

Second, to facilitate comparison with other methods, we repartition the dataset into training, validation, and testing subsets with a ratio of 70:13:17. Then, we retrain the AMI-NanoNet by using the same parameter settings as in the previous step, and select the model with the highest accuracy on the validation set for the subsequent experiments. Finally, we can achieve an accuracy of 93.8650% on the partitioned validation set, and this model can attain an accuracy of 95.1163% on the testing set.

However, due to the small number of categories in TrashNet dataset and the absence of a multilevel waste categorization system, the partitioning method depicted in Fig. 6 is not applicable to such a dataset. To address this issue, we expand the definition of Level 1 to include multiple random categories that encompass six Level 2 categories in TrashNet, with the requirement that the number of Level 1 categories be fewer than 6. This definition is based on the fact that deep learning algorithms rely on backpropagation of category labels. Similar to the concept of changing prompts in natural language processing, modifying the definition and scope of category labels can alter the distribution of parameters within a deep learning algorithm and ultimately impact its understanding of image features.

To minimize the impact of class imbalance on the algorithm, we grouped cardboard, metal, and plastic into one category, and glass, paper, and trash into another. This results in a nearly 1:1 ratio of the number of images for the two categories belonging to Level 1. Due to the considerably smaller size of the TrashNet dataset in comparison to PlantVillage, we relax the constraint

TABLE VIII
PERFORMANCE COMPARISON OF MODELS IN TRASHNET

Methods	Accuracy(%)	Total Images	GFLOPs	Params(M)
MobileNet [64]	84.00	2527(70% training, 17% testing, 13% valid)	0.55	4.1
EfficientNet-B0 [64]	90.02	2527(70% training, 17% testing, 13% valid)	0.39	5.3
EfficientNet-B1 [64]	91.53	2527(70% training, 17% testing, 13% valid)	0.69	7.8
Pre-trained ResNet50 [64], [65]	91.40	2527(90% training, 10% testing)	3.86	23.5
ResNet50 + SE [64], [65]	91.88	2527(70% training, 17% testing, 13% valid)	3.87	26.1
ResNet50 + RecycleNet [64]	94.20	2527(70% training, 17% testing, 13% valid)	-	24.2
MobileNetV2 [66]	91.42	2527(80% training, 10% testing, 10% valid)	0.32	3.5
EnCNN-UPMWS [66]	94.11	2527(80% training, 10% testing, 10% valid)	>5.69	>35.7
ResNet18 + SMM [67]	95.87	2527(70% training, 30% testing)	>1.82	>11.2
ViT-B/32 [64]	96.98	2527(70% training, 17% testing, 13% valid)	8.56	88.3
ResNet101 + EfficientNet-B0 + EfficientNet-B1 [68]	93.50	2527(50% training, 25% testing, 25% valid)	>8.93	>57.7
Proposed Method	95.58	2527(70% training, 17% testing, 13% valid)	0.34	1.07

($\alpha' = 93.8650\% - 0.03\%$) on the number of iterations for each learner. In addition, some modifications to the algorithm parameters are detailed in Table VI. It should be noted that the adjustment of the number of enhanced images per error is also based on this reason. Moreover, using different enhancement quantities for the two learners can further reduce the impact of category imbalance on our algorithm.

Finally, we achieved the final results of ensemble learning after six iterations. Our model can achieve the accuracy of 94.7853% on the validation set and the accuracy of 95.5814% on the test set. The comparison with other advanced classification methods is listed in Table VIII. Due to the variations in the indicators reported in different literature, we use GFLOPs and parameters as approximations to describe the inference speed of the models in this comparison. These methods mainly come from state-of-the-art classical image classification models, lightweight image classification models, and ensemble learning-based image classification models.

It is clear that our method is still the one with the lowest number of parameters among these methods. Compared with high-accuracy methods, our model has significantly fewer parameters and GFLOPs. Compared with methods with low computational complexity and parameter count, our method achieves significantly higher accuracy. Moreover, compared with other ensemble learning methods [66], [68] performed on this dataset, our model outperforms these methods in both accuracy and inference speed.

In addition, the effectiveness of our ensemble learning method is demonstrated in Fig. 14. The continuously left-shifted curve indicates the validity of the expansion of our definition in the Level 1 category. Besides, even though the mean errors are relatively high, after iterative training and voting, the accuracy of learner A_1 and learner A_2 increased by 1.1628% and 2.0930% on the test set, respectively. It further demonstrates the effectiveness of our algorithm. These results demonstrate that our ensemble learning method can be applied to various image classification domains.

V. DISCUSSION

In the field of plant disease diagnosis, drones and mobile automatic detection equipment will become the main means of machine diagnosis of plant diseases in the future. A model better

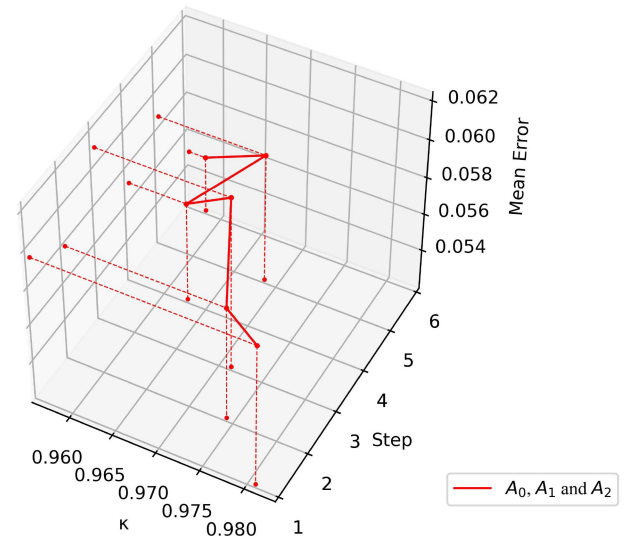


Fig. 14. Variation of the difference among these learners in iterations by using the κ statistic and mean error on TrashNet dataset.

suiting to low-power devices would be more beneficial for the task of plant disease diagnosis in a variety of different situations.

In this study, we tried a number of different approaches to improve the network's ability to diagnose plant diseases. Within these approaches, the AMI module and FGCL training strategy offer a model that balances recognition accuracy and inference speed for low-power devices in real-world agricultural scenarios. Furthermore, extensive experimental validation has been conducted to support their effectiveness. However, in real-world agricultural scenarios, the complexity of plant disease features and image backgrounds far exceed what is contained within existing datasets. Even with various image augmentation techniques employed to create as many disease features as possible, lightweight networks based on supervised learning still require transfer learning and fine-tune strategies to address the large range of feature variations encountered in practical settings. Including the construction of a more extensive dataset compared with the PlantVillage dataset, we plan to explore these attempts in future research.

On the other hand, optimizing deep learning models for both inference speed and recognition accuracy at the same time

is a difficult challenge. Jiang et al. [69] pointed out that the performance of CNN is mainly related to its structure, and second related to the training strategy. Our experimental results are consistent with this view. However, for the problems we need to solve, the training strategy is more important than the structure of the CNN. To address this challenge, we proposed the FGCL strategy and a lightweight ensemble learning method aiming to improve recognition accuracy without compromising the model's inference speed. However, these methods introduce a number of hyperparameters that need to be adjusted during practical training, which posing obstacles to their application in real-world agricultural scenarios. Further optimization of these methods and reduction in the number of hyperparameters should be explored through experimental investigations in the future.

Finally, in our ensemble learning algorithm, increasing the diversity among learners, and reducing mean error are often a pair of conflicting concepts. This poses challenges to further improving the recognition accuracy of the model. In our view, exploring the construction of learners and the iteration conditions of this algorithm is an important direction for improving the ensemble performance. We will continue to try these ideas in future research.

VI. CONCLUSION

In this study, in order to obtain a real-time identification network of plant diseases suitable for low-power devices, we tried from three aspects: network structure, training strategy, and ensemble learning. First, we proposed AMI modules A and B based on Roofline theory, and further proposed the AMI-NanoNet. This structure can be trained to high accuracy with a small number of convolutional layers and a small number of parameters. Then, in order to further upgrade the accuracy of the network, we improved the original training strategy based on the curriculum learning concept and proposed the FGCL strategy. Tested in ablation experiments, our new training strategy has higher stability and better performance. Moreover, we proposed to use ensemble learning to further improve the accuracy of our model, and we only ensembled the classifier part of the network. The resulting model based on these methods achieved an inference speed of 30.46 ms per image on Jetson Nano, while attaining an identification accuracy of 99.8708% on the test set. In the comparison with other cutting-edge methods, the experimental results show that our method has reached the state-of-the-art level by combining the three indicators of inference speed, parameter size, and accuracy. Besides, we validated our proposed methods on the TrashNet dataset to demonstrate its applicability in other scenarios. By expanding the definition of Level 1, our method can still be applied to classification datasets outside the realm of plant diseases recognition, and demonstrates competitiveness compared with other existing methods.

The automation of plant disease recognition is currently in an exploratory phase. For low-power devices and lightweight recognition networks, training broader data samples, alleviating the debugging challenges posed by excessive hyperparameters, and standardizing the model's understanding of disease features have emerged as crucial considerations for the application of AI

methods in real-world agricultural scenarios. In the future, we will undertake broader research efforts to further advance the application of AI methods in agricultural settings.

REFERENCES

- [1] A. M. Alvarez, "Integrated approaches for detection of plant pathogenic bacteria and diagnosis of bacterial diseases," *Annu. Rev. Phytopathol.*, vol. 42, pp. 339–366, 2004.
- [2] Z. Li et al., "Non-invasive plant disease diagnostics enabled by smartphone-based fingerprinting of leaf volatiles," *Nature Plants*, vol. 5, no. 8, pp. 856–866, Aug. 2019.
- [3] S. H. Lee, C. S. Chan, S. J. Mayo, and P. Remagnino, "How deep learning extracts and learns leaf features for plant classification," *Pattern Recognit.*, vol. 71, pp. 1–13, 2017.
- [4] J. Liu and X. Wang, "Plant diseases and pests detection based on deep learning: A review," *Plant Methods*, vol. 17, no. 1, 2021, Art. no. 22.
- [5] N. Kundu et al., "IoT and interpretable machine learning based framework for disease prediction in pearl millet," *Sensors*, vol. 21, no. 16, 2021, Art. no. 5386.
- [6] T. P. Wallace, O. R. Mitchell, and K. Fukunaga, "Three-dimensional shape analysis using local shape descriptors," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-3, no. 3, pp. 310–323, May 1981.
- [7] P. Sharma and S. Sharma, "Paradigm shift in plant disease diagnostics: A journey from conventional diagnostics to nano-diagnostics," in *Current Trends in Plant Disease Diagnostics and Management Practices*, P. Kumar, V. K. Gupta, A. K. Tiwari, and M. Kamle, Eds. Cham, Switzerland: Springer, 2016, pp. 237–264.
- [8] Z. Li, T. Yu, R. Paul, J. Fan, Y. Yang, and Q. Wei, "Agricultural nanodiagnoses for plant diseases: Recent advances and challenges," *Nanoscale Adv.*, vol. 2, no. 8, pp. 3083–3094, 2020.
- [9] X. Liu, W. Min, S. Mei, L. Wang, and S. Jiang, "Plant disease recognition: A large-scale benchmark dataset and a visual region and loss reweighting approach," *IEEE Trans. Image Process.*, vol. 30, pp. 2003–2015, 2021.
- [10] W. Ge, X. Lin, and Y. Yu, "Weakly supervised complementary parts models for fine-grained image classification from the bottom up," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 3034–3043.
- [11] Q. Liang, S. Xiang, Y. C. Hu, G. Coppola, D. Zhang, and W. Sun, "PD2SE-Net: Computer-assisted plant disease diagnosis and severity estimation network," *Comput. Electron. Agriculture*, vol. 157, pp. 518–529, Feb. 2019.
- [12] M. Yebasse, B. Shimelis, H. Warku, J. Ko, and K. J. Cheoi, "Coffee disease visualization and classification," *Plants*, vol. 10, no. 6, 2021, Art. no. 1257.
- [13] P. Saleem and A. Mahmood, "Plant disease detection and classification by deep learning," *Plants*, vol. 8, no. 11, 2019, Art. no. 468.
- [14] J. Chen, H. Yin, and D. Zhang, "A self-adaptive classification method for plant disease detection using GMDH-Logistic model," *Sustain. Comput. Inform. Syst.*, vol. 28, Dec. 2020, Art. no. 100415.
- [15] P. Wang, T. Niu, Y. Mao, Z. Zhang, B. Liu, and D. He, "Identification of apple leaf diseases by improved deep convolutional neural networks with an attention mechanism," *Front. Plant Sci.*, vol. 12, no. 1997, Sep. 2021, Art. no. 723294.
- [16] L. Bi and G. Hu, "Improving image-based plant disease classification with generative adversarial network under limited training set," *Front. Plant Sci.*, vol. 11, 2020, Art. no. 583438.
- [17] A. T. S. Kalvakolanu, "Plant disease detection from images," 2020, *arXiv:2003.05379*.
- [18] S. Vallabhajosyula, V. Sistla, and V. K. K. Kolli, "Transfer learning-based deep ensemble neural network for plant leaf disease detection," *J. Plant Dis. Protection*, vol. 129, no. 3, pp. 545–558, 2022.
- [19] M. Khanramaki, E. A. Asli-Ardeh, and E. Kozegar, "Citrus pests classification using an ensemble of deep learning models," *Comput. Electron. Agriculture*, vol. 186, Jul., 2021, Art. no. 106192.
- [20] M. Türkoğlu and D. Hanbay, "Plant disease and pest detection using deep learning-based features," *Turkish J. Elect. Eng. Comput. Sci.*, vol. 27, no. 3, pp. 1636–1651, 2019.
- [21] S. Xiang, Q. Liang, W. Sun, D. Zhang, and Y. Wang, "L-CSMS: Novel lightweight network for plant disease severity recognition," *J. Plant Dis. Protection*, vol. 128, no. 2, pp. 557–569, 2021.
- [22] J. Chen, D. Zhang, and Y. A. Nanehkan, "Identifying plant diseases using deep transfer learning and enhanced lightweight network," *Multimedia Tools Appl.*, vol. 79, no. 41/42, pp. 31497–31515, 2020.

- [23] A. Bhujel, N.-E. Kim, E. Arulmozhi, J. K. Basak, and H.-T. Kim, "A lightweight attention-based convolutional neural networks for tomato leaf disease classification," *Agriculture*, vol. 12, no. 2, 2022, Art. no. 228.
- [24] P. Wang et al., "Fine-grained grape leaf diseases recognition method based on improved lightweight attention network," *Front. Plant Sci.*, vol. 12, Oct. 2021, Art. no. 738042.
- [25] V. Gonzalez-Huitron, J. A. Lón-Borges, A. E. Rodriguez-Mata, L. E. Amabilis-Sosa, B. Ramírez-Pereda, and H. Rodriguez, "Disease detection in tomato leaves via CNN with lightweight architectures implemented in raspberry Pi 4," *Comput. Electron. Agriculture*, vol. 181, Feb., 2021, Art. no. 105951.
- [26] K. Kc, Z. Yin, M. Wu, and Z. Wu, "Depthwise separable convolution architectures for plant disease classification," *Comput. Electron. Agriculture*, vol. 165, 2019, Art. no. 104948.
- [27] W. Chen, J. Chen, R. Duan, Y. Fang, Q. Ruan, and D. Zhang, "MS-DNet: A mobile neural network for plant disease identification," *Comput. Electron. Agriculture*, vol. 199, 2022, Art. no. 107175.
- [28] J. Chen, W. Chen, A. Zeb, S. Yang, and D. Zhang, "Lightweight inception networks for the recognition and detection of rice plant diseases," *IEEE Sensors J.*, vol. 22, no. 14, pp. 14628–14638, Jul. 2022.
- [29] J. Chen, X. Deng, Y. Wen, W. Chen, A. Zeb, and D. Zhang, "Weakly-supervised learning method for the recognition of potato leaf diseases," *Artif. Intell. Rev.*, vol. 56, pp. 7985–8002, 2023.
- [30] S. Williams, A. Waterman, and D. Patterson, "Roofline," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [31] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical guidelines for efficient CNN architecture design," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 122–138.
- [32] NVIDIA Developer - Jetson nano, 2019. Accessed: Feb. 2, 2023. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-nano>
- [33] S. A. Tsafaris, M. Minervini, and H. Scharr, "Machine learning for plant phenotyping needs image processing," *Trends Plant Sci.*, vol. 21, no. 12, pp. 989–991, 2016.
- [34] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.
- [35] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics through machine learning and crowdsourcing," 2015, *arXiv:1511.08060*.
- [36] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, "PlantDoc: A dataset for visual plant disease detection," in *Proc. the 7th ACM IKDD CoDS 25th COMAD*, 2020, pp. 249–253.
- [37] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Front. Plant Sci.*, vol. 7, 2016, Art. no. 1419.
- [38] E. C. Too, L. Yujian, S. Njuki, and L. Yingchun, "A comparative study of fine-tuning deep learning models for plant disease identification," *Comput. Electron. Agriculture*, vol. 161, pp. 272–279, Jun. 2019.
- [39] M. A. B. Mahmoud, P. Guo, and K. Wang, "Pseudoinverse learning autoencoder with DCGAN for plant diseases classification," *Multimedia Tools Appl.*, vol. 79, no. 35/36, pp. 26245–26263, 2020.
- [40] Y. A. Nanehkaran, D. Zhang, J. Chen, Y. Tian, and N. Al-Nabhan, "Recognition of plant leaf diseases based on computer vision," *J. Ambient Intell. Hum. Comput.*, vol. 11, no. 4, pp. 1–18, 2020.
- [41] A. Almadhor, H. T. Rauf, M. I. U. Lali, R. Damaševičius, B. Alouffi, and A. Alharbi, "AI-Driven framework for recognition of guava plant diseases through machine learning from DSLR camera sensor based high resolution imagery," *Sensors*, vol. 21, no. 11, 2021, Art. no. 3830.
- [42] J. Chen, D. Zhang, Y. A. Nanehkaran, and D. Li, "Detection of rice plant diseases based on deep transfer learning," *J. Sci. Food Agriculture*, vol. 100, no. 7, pp. 3246–3256, 2020.
- [43] M. H. Saleem, J. Potgieter, and K. M. Arif, "Plant disease classification: A comparative evaluation of convolutional neural networks and deep learning optimizers," *Plants*, vol. 9, no. 10, 2020, Art. no. 1319.
- [44] Ü. Atila, M. Uçar, K. Akylol, and E. Uçar, "Plant leaf disease classification using EfficientNet deep learning model," *Ecological Informat.*, vol. 61, 2021, Art. no. 101182.
- [45] M. X. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn. Res.*, Long Beach, CA, USA, 2019, pp. 6105–6114.
- [46] D. O. Oyewola, E. G. Dada, S. Misra, and R. Damaševičius, "Detecting cassava mosaic disease using a deep residual convolutional neural network with distinct block processing," *PeerJ Comput. Sci.*, Mar., vol. 7, 2021, Art. no. e352.
- [47] R. Gao, R. Wang, L. Feng, Q. Li, and H. Wu, "Dual-branch, efficient, channel attention-based crop disease identification," *Comput. Electron. Agriculture*, vol. 190, 2021, Art. no. 106410.
- [48] W. Albattah, M. Nawaz, A. Javed, M. Masood, and S. Albahli, "A novel deep learning method for detection and classification of plant diseases," *Complex Intell. Syst.*, vol. 8, no. 1, pp. 507–524, 2022.
- [49] B. T. Hanh, H. Van Manh, and N.-V. Nguyen, "Enhancing the performance of transferred efficientnet models in leaf image-based plant disease classification," *J. Plant Dis. Protection*, vol. 129, no. 3, pp. 623–634, 2022.
- [50] P. Bansal, R. Kumar, and S. Kumar, "Disease detection in apple leaves using deep convolutional neural network," *Agriculture*, vol. 11, no. 7, 2021, Art. no. 617.
- [51] J. Chen, A. Zeb, Y. A. Nanehkaran, and D. Zhang, "Stacking ensemble model of deep learning for plant disease recognition," *J. Ambient Intell. Hum. Comput.*, vol. 14, pp. 12359–12372, 2023.
- [52] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.
- [53] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE 31th Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4510–4520.
- [54] C. Yang, Y. Wang, T. Kurth, S. Farrell, and S. Williams, "Hierarchical roofline performance analysis for deep learning applications," in *Proc. Intell. Comput. - Proc. Comput. Conf.*, 2021, pp. 473–491.
- [55] E. A. H. Mora, V. Gonzalez-Huitron, A. E. Rodriguez-Mata, and H. R. Rangel, "Convolutional neural networks-based plant disease detection implemented on low-power consumption device," in *Proc. IEEE Int. Autumn Meeting Power, Electron, Comput.*, 2021, pp. 1–6.
- [56] Z. Chen, F. Zheng, Q. Yu, R. Sun, F. Guo, and Z. Chen, "Evaluating performance of AI operators using roofline model," *Appl. Intell.*, vol. 52, no. 7, pp. 7054–7069, 2022.
- [57] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional block attention module," in *Proc. 15th Eur. Conf. Comput. Vis.*, 2018, pp. 3–19.
- [58] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE 30th Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1800–1807.
- [59] P. Tang, X. Yan, Q. Liang, and D. Zhang, "AFLN-DGCL: Adaptive feature learning network with difficulty-guided curriculum learning for skin lesion segmentation," *Appl. Soft Comput.*, vol. 110, 2021, Art. no. 107656.
- [60] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proc. 26th Annu. Int. Conf. Mach. Learn.*, 2009, pp. 41–48.
- [61] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Inf. Process. Manage.*, vol. 45, no. 4, pp. 427–437, 2009.
- [62] S. Mao, L. C. Jiao, L. Xiong, and S. Gou, "Greedy optimization classifiers ensemble based on diversity," *Pattern Recognit.*, vol. 44, no. 6, pp. 1245–1261, 2011.
- [63] G. Thung and M. Yang, "Classification of trash for recyclability status," Stanford Univ., Stanford, CA, USA, Tech. Rep. CS229, 2016. [Online]. Available: <http://cs229.stanford.edu/proj2016/poster/ThungYangClassificationOfTrashForRecyclabilityStatus-poster.pdf>
- [64] K. Huang, H. Lei, Z. Jiao, and Z. Zhong, "Recycling waste classification using vision transformer on portable device," *Sustainability*, vol. 13, no. 21, 2021, Art. no. 11572.
- [65] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 7132–7141.
- [66] H. Zheng and Y. Gu, "EnCNN-UPMWS: Waste classification by a CNN ensemble using the UPM weighting strategy," *Electronics*, vol. 10, no. 4, 2021, Art. no. 427.
- [67] Q. Zhang et al., "Recyclable waste image recognition based on deep learning," *Resour. Conserv. Recyclable*, vol. 171, 2021, Art. no. 105636.
- [68] M.-H. Huynh, P.-T. Pham-Hoai, A.-K. Tran, and T.-D. Nguyen, "Automated waste sorting using convolutional neural network," in *Proc. 7th NAFOSTED Conf. Inf. Comput. Sci.*, Ho Chi Minh City, Vietnam, 2020, pp. 102–107.
- [69] J. Jiang et al., "Evaluation of diverse convolutional neural networks and training strategies for wheat leaf disease identification with field-acquired photographs," *Remote Sens.*, vol. 14, no. 14, 2022, Art. no. 3446.